

Graph Data Collection and Construction

Ryan Pek

April 25, 2026

1 Overview

The Singapore bus network is modelled as a directed graph:

$$G = (V, E)$$

where:

- V is the set of bus stops
- $E \subseteq V \times V$ is the set of valid directed transitions

An edge (u, v) exists if there exists a bus service s and direction d such that v is the immediate successor of u in the ordered route sequence of (s, d) .

The objective of this stage is to construct G from raw LTA DataMall data.

2 Data Sources

The following endpoints are used:

- **BusStops**: metadata (stop code, description, coordinates)
- **BusRoutes**: ordered stop sequences for each service
- **BusServices**: service-level metadata
- **BusArrival**: used to determine service presence at stops

All responses are stored as raw JSON snapshots without modification.

2.1 Pagination and Empirical Limits

All endpoints return data in pages of up to 500 records, accessed via:

$$?\$skip=n \cdot 500$$

where n is the page index.

Empirically observed payload counts (as of last collection):

- **BusStops**: ≈ 11 pages
- **BusRoutes**: ≈ 51 pages
- **BusServices**: ≈ 2 pages

In implementation, one additional page is queried beyond these values to detect changes in dataset size.

These values are not assumed fixed and should be treated as lower bounds.

3 Data Pipeline

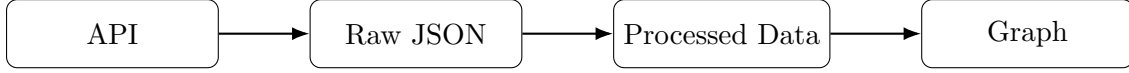


Figure 1: Data flow

4 Construction

4.1 Vertex Set

The vertex set is constructed as:

- union of all `BusStopCode` from `BusStops`
- union of all `BusStopCode` from `BusRoutes`
- deduplicated

This ensures coverage in the presence of incomplete or inconsistent datasets.

4.2 Route Extraction

For each service:

- group entries by `ServiceNo`
- split by `Direction`
- entries are sorted by `StopSequence` to recover traversal order

Each service produces up to two ordered sequences:

$$R_{s,1}, \quad R_{s,2}$$

4.3 Edge Construction

Given a route:

$$R = [v_1, v_2, \dots, v_k]$$

edges are constructed as:

$$(v_1, v_2), (v_2, v_3), \dots, (v_{k-1}, v_k)$$

The final edge set is the union over all services and directions.

Duplicate edges are collapsed into a single adjacency entry.

4.4 Adjacency Representation

The graph is stored as an adjacency map:

```
{
  "85049": {
    "neighbours": [...],
    "distfromint": ...,
    "timesvisited": 0
  }
}
```

4.5 Node Fields

- **neighbours**: outgoing edges
- **distfromint**: distance from route origin
- **timesvisited**: mutable traversal metadata used by search algorithms

5 Properties

- directed graph
- contains cycles (loop services)
- not guaranteed strongly connected
- multiple services may induce identical edges

6 Edge Cases

6.1 Terminal Stops

Final stops in a route have no outgoing edges.

6.2 Loop Services

Loop services (e.g. 225G / 225W):

- form cycles
- must be treated as separate directed paths

6.3 Self-loops

Certain stops may incorrectly reference themselves as neighbours due to data anomalies.

These are explicitly filtered during graph construction.

6.4 Data Inconsistencies

Observed issues:

- stops present in routes but missing from stops dataset
- services appearing in arrival data but not in route data
- occasional malformed neighbour relationships

These are handled manually or filtered during processing.

7 Implementation Notes

The system is implemented in two stages:

- Python prototype:
 - data collection
 - graph construction
- Go rewrite:
 - API layer (HTTP fetch)
 - storage layer (JSON + filesystem)
 - graph construction layer